Republic of the Philippines
CARAGA STATE UNIVERSITY
CABADBARAN CAMPUS
T. Curato St., Cabadbaran City, Agusan del Norte
Competence Service Uprightness

COLLEGE OF ENGINEERING AND INFORMATION TECHNOLOGY

LABORATORY MANUAL ACTIVITY

Planning the Flutter Web–REST API–PostgreSQL Technology Stack of Your Own Project

Flutter Framework and Technology Stack

Course	IT 110 – Web Systems and Technologies
Activity	Laboratory Activity No. 3
Output	Flutter Web–REST API–PostgreSQL technology stack blueprint, stack-role worksheets, and group presentation
Maximum Score	80 points

Student / Project Information

Block: BSIT - 3A1 IBPH3	Date: September 17, 2026
System Title: Humanitarian Aid Distribution Management System with Machine Learning-Based Household Vulnerability Classification	Instructor: Jean Y. Villasin, MIT
Members: Steven P. Payapaya **Kyle M. Borja**	

1. Activity Alignment and Purpose

Topic Learning Outcome: Explain the prescribed Flutter Web–REST API–PostgreSQL technology stack and justify the role of each component in the proposed web system.

Course Outcome Alignment: CO1 – Explain web architecture and the roles of client-side, server-side, framework, API, database, and deployment components in the end-to-end flow of a web system.

Syllabus Assessment: Flutter–API–PostgreSQL Technology Stack Blueprint; stack-role explanation; online architecture justification task.

Purpose of the Activity

Activity 2 established the overall architecture of your semester project. In Activity 3, your team will convert that architecture into the prescribed implementation stack: Flutter Web for the client interface, a REST API using an instructor-approved backend framework/runtime for application logic, and PostgreSQL as the persistence layer developed in IT 108. You will also identify the role of Dart, Flutter packages, Git/GitHub, and the deployment environment so the blueprint can guide later implementation activities.

Important distinction: Flutter Web is the client application layer; the REST API is the application and integration layer; PostgreSQL is the persistence layer. The Flutter client must not contain database credentials or issue raw SQL directly to PostgreSQL in the prescribed stack.

2. Intended Learning Objectives

- Distinguish a framework, library/package, runtime, API style/protocol, DBMS, version-control tool, and deployment platform.
- Explain the responsibilities of Flutter Web, the REST API/backend, and PostgreSQL in the team's proposed system.
- Identify the Dart/Flutter fundamentals, widgets, packages, state-management approach, and client-side validation needed by major project features.
- Select and justify an instructor-approved backend framework/runtime for the REST API without shifting database or client responsibilities into the wrong layer.
- Map REST API resources and operations to the PostgreSQL tables, relationships, constraints, and transactions being developed in IT 108.
- Create a system-specific Flutter Web–REST API–PostgreSQL technology stack blueprint that includes Git/GitHub, security boundaries, and deployment targets.
- Explain and defend why each technology belongs in the stack and how the components communicate.

3. Activity Specifications

Activity Item	Requirement
Mode	Existing semester-project group; all members must contribute to the technology decisions and be able to explain one assigned stack component.
Suggested Time	One laboratory session, approximately 3 hours, plus the scheduled group presentation.
Primary Output	One project-specific Flutter Web–REST API–PostgreSQL Technology Stack Blueprint.

Activity Item	Requirement
Supporting Output	Technology classification worksheet, stack-role matrix, Flutter client plan, REST API plan, PostgreSQL mapping, Git/GitHub and deployment plan, 120–180 word stack-role explanation, and group presentation.
Recommended Tools	diagrams.net/Draw.io, Lucidchart, Figma, Canva, PowerPoint, Git/GitHub, Flutter/Dart documentation, backend API documentation, and PostgreSQL documentation.
Presentation Format	5–7 minutes per group followed by 2–3 minutes of questions. Every member must explain an assigned technology, interface, or integration decision.

4. Pre-Laboratory Requirements

- Final or revised architecture diagram from Laboratory Activity No. 2.
- Approved system title, primary users/roles, and at least five major system features.
- Current ERD or normalized PostgreSQL schema from IT 108, including primary keys, foreign keys, and important constraints.
- Instructor-approved backend framework/runtime for the REST API. If approval is still pending, bring no more than two candidate options for consultation.
- Initial list of Flutter packages/state-management tools and external services that the project genuinely needs.
- Access to the project GitHub repository and the team's planned deployment environment or hosting options.
- Copies of Flutter/Dart, REST API/backend, and PostgreSQL documentation relevant to the selected technologies.

5. Part A – Technology Classification and Role Check

Before assigning technologies to the project, classify each item correctly. Use the following distinctions: **framework** provides an application structure and conventions; **library/package** provides reusable functions or components; **runtime** executes application code; and the remaining categories describe communication, persistence, version control, or hosting responsibilities.

Category	Meaning
Framework	Provides structure, conventions, lifecycle, and reusable building blocks for an application.
Library / package	Reusable code added to the application for a focused capability such as HTTP, state management, charts, maps, or validation.
Runtime	Environment that executes application or server code.
API style / protocol / data format	Defines how components communicate and how requests/responses are represented.
DBMS	Manages persistent structured data, constraints, transactions, and queries.
Version control / collaboration	Tracks changes, supports branches and reviews, and stores project history.
Deployment platform / environment	Runs or hosts the client, API, database, and supporting services.

Classification Worksheet

Technology / Concept	Classification	Role in Your Project	Project-Specific Choice / Evidence
Dart	Programming Language	Used to develop the Flutter Web application and handle the client-side logic.	Dart used as the programming language for the Flutter Web client.
Flutter	Framework	Provides the structure, widgets, screens, navigation, forms, and user interface of the system.	Flutter Web used for the client application, including login, household registration, assessment, vulnerability results, aid distribution, and reports.
Flutter package for HTTP/API calls	Library / Package	Sends requests to the REST API and receives responses from the backend.	HTTP/API client package used by Flutter Web to communicate with the REST API through HTTP requests and JSON responses.
State-management package or pattern	Library / Package / Pattern	Manages client-side state, user actions, form data, loading states, and displayed results.	Flutter State Management / Controllers used to manage user input, form data, navigation, loading states, and displayed results.
REST + HTTPS + JSON	API style / protocol / data format	Allows Flutter Web and the REST API to communicate securely using HTTP requests and JSON data.	REST API over HTTPS using JSON used for client-to-backend requests and responses.
Backend language	Python	Language used to develop the server-side application logic of the REST API.	Python used for the REST API backend.
Backend runtime	Runtime	Executes the backend application code and REST API.	Python Runtime runs the FastAPI backend application.
PostgreSQL	DBMS	Stores and manages user, household, assessment, vulnerability, aid, and distribution records.	PostgreSQL used to store user accounts, household information, assessments, vulnerability classifications, aid records, and distribution records.
Git / GitHub	Version control / collaboration	Tracks project changes and supports collaboration among group members.	Git + GitHub used for source-code version control, collaboration, commits, and project history.
Deployment platform(s)	Deployment platform / environment	Hosts the Flutter Web client, REST API, PostgreSQL database, and supporting ML service.	Deployment environment to be finalized by the group for the Flutter Web application, FastAPI backend, PostgreSQL database, and Machine Learning service.

6. Part B – Prescribed Stack Role Matrix

Complete the matrix using the exact technologies planned for your system. The “Must Not Be Responsible For” column is included to prevent duplicated logic and insecure shortcuts.

Layer / Area	Prescribed Technology	Your Project-Specific Choice	Main Responsibility	Must Not Be Responsible For
Presentation / Client	Flutter Web + Dart	Flutter Web + Dart	Screens, widgets, navigation, state, form handling, user feedback	Database credentials; raw SQL; final authorization rules
Client packages	Team-selected Flutter packages	Flutter HTTP/API package + Flutter State Management / Controllers	HTTP/API calls, state management, validation, maps/charts, or other approved client functions	Server-only secrets or database administration
Integration	REST API over HTTPS using JSON	REST API over HTTPS using JSON	Defines endpoints, request/response formats, status codes, and client–server communication	Persistent data storage by itself
Application / Backend	FAST API / Python	FAST API / Python	Authentication/authorization checks, validation, business rules, orchestration, API response generation	Rendering the Flutter UI; storing production data in local code files
Persistence	PostgreSQL schema developed in IT 108	PostgreSQL	Tables, relationships, constraints, transactions, queries, durable records	User-interface logic or route navigation

Version control	Git + GitHub	Git + GitHub	Source history, collaboration, branches, pull requests, issue/document tracking	Running the production system
Deployment	Team-selected hosting / server / managed DB	To be finalized by the group.	Runs Flutter Web, API, PostgreSQL, and environment configuration	Replacing application-layer security or database constraints

Stack-role rule

A good blueprint separates concerns. Flutter Web handles the user experience and client state; the REST API/backend protects and processes requests; PostgreSQL persists validated data and enforces relational integrity. Git/GitHub supports collaboration, while deployment platforms host the running components. If a responsibility appears in two layers, the team must explain why it is not duplicated or contradictory.

7. Part C – Flutter Web Client Blueprint

Plan the client side around your actual system features. List at least five important screens or feature modules. Use package names only when the team has decided to use them.

Feature / Screen	Main Widgets / Forms	State / Controller	Flutter Package(s)	API Operation Needed	Client Validation / User Feedback
Login	Login Form, TextFields, Login Button	Login State / Controller	Flutter HTTP/API client package	POST Login	Required username/password; show error message for invalid login; show loading while logging in
Household Registration	Registration Form, TextFields, Dropdowns, Submit Button	Household Registration State / Controller	Flutter HTTP/API client package	POST Household	Required fields; validate entered household information; show success/error message
Household Assessment	Assessment Form, TextFields, Dropdowns, Submit Button	Assessment State / Controller	Flutter HTTP/API client package	POST Assessment	Validate required assessment fields; show validation errors; show loading while submitting
Vulnerability Results	Result Card, Text, Status/Level Display	Vulnerability Result State / Controller	Flutter HTTP/API client package	GET Vulnerability Result	Show loading while retrieving result; show result or error message
Beneficiary List	ListView/Data Table, Search/Filter Controls	Beneficiary List State / Controller	Flutter HTTP/API client package	GET Beneficiaries	Show empty state if no beneficiaries; show loading/error feedback
Aid Distribution	Aid Distribution Form, Dropdowns, TextFields, Submit Button	Aid Distribution State / Controller	Flutter HTTP/API client package	POST Aid Distribution	Validate required fields; confirm successful recording; show error if submission fails
Reports	Report List/Table, Filter Controls, Summary Widgets	Reports State / Controller	Flutter HTTP/API client package	GET Reports	Show loading state; show empty state when no records are available; show error feedback

Client planning checklist:

- Routing/navigation between screens is identified.
- Responsive web layout and loading/error/empty states are considered.
- Client-side validation improves usability but does not replace server-side validation.
- No database password, private API key, or other server secret is embedded in Flutter Web source code.

8. Part D – REST API and Backend Plan

Selected backend framework: FastAPI

Selected backend runtime: Python

Why this choice fits the project (3–5 sentences):

The backend will provide the REST API that connects the Flutter Web client to the PostgreSQL database. It will handle authentication, validation, business rules, and requests from the Flutter application. It will also communicate with the Machine Learning Classification Service for household vulnerability classification. This setup keeps the database and server-side logic separate from the Flutter Web client.

Minimum API Contract Worksheet

Define at least six REST operations that support the project's real workflows. Use an instructor-approved endpoint naming convention and appropriate HTTP methods.

Use Case	Method	Endpoint	Auth / Role	Request Data	Server Action / Rule	Response / Status
User Login	POST	/api/login	Public	Username, password	Authenticate user and check account credentials	User info/token / 200 OK; invalid credentials / 401 Unauthorized
Register Household	POST	/api/households	LGU Administrator / LGU Staff	Household information	Validate and save household information in PostgreSQL	Household created / 201 Created
Submit Assessment	POST	/api/assessments	LGU Administrator / LGU Staff	Household ID, assessment data	Validate assessment and save assessment record	Assessment saved / 201 Created
Get Vulnerability Result	GET	/api/households/{id}/vulnerability	LGU Administrator / LGU Staff	Household ID	Retrieve/request vulnerability classification from ML service	Vulnerability result / 200 OK
View Beneficiary List	GET	/api/beneficiaries	LGU Administrator / LGU Staff	Optional filters	Retrieve beneficiary records based on available assessment/classification data	Beneficiary list / 200 OK
Record Aid Distribution	POST	/api/aid-distributions	LGU Administrator / LGU Staff	Household/beneficiary ID, aid details	Validate and record the aid distribution	Distribution recorded / 201 Created
View Reports	GET	/api/reports	LGU Administrator / LGU Staff	Report/filter parameters	Retrieve relevant distribution and beneficiary information for reports	Report data / 200 OK

Map the API resources to the PostgreSQL schema being designed in IT 108. Do not invent a second unrelated database model for IT 110. Record important keys, constraints, and transaction rules that affect the API.

API Resource / Feature	PostgreSQL Table(s)	PK / FK / Constraint	DML Operation	Transaction / Integrity Rule	Returned Fields / Result
User Login	users	Primary key; unique username; required credentials	SELECT	Only valid user credentials can be authenticated	User ID, username, role
Register Household	households	Primary key; required household information	INSERT	Household information must be valid before saving	Household ID, household information
Submit Assessment	assessments	Primary key; household relationship	INSERT	Assessment must reference an existing household	Assessment ID, assessment data
Vulnerability Classification	vulnerability_classifications	Primary key; household/assessment relationship	INSERT	Classification must belong to an existing household and valid assessment	Classification ID, vulnerability level
View Vulnerability Result	vulnerability_classifications	Primary key; household relationship	SELECT	Result must belong to the requested household	Vulnerability level and classification data
View Beneficiary List	households, vulnerability_classifications	Related keys and constraints	SELECT	Only existing and valid household records are retrieved	Beneficiary/household information and vulnerability level
Record Aid Distribution	aid_distributions	Primary key; household/beneficiary relationship	INSERT	Aid distribution must reference an existing household/beneficiary	Aid distribution ID, aid details, distribution date
View Reports	households, assessments, vulnerability_classifications, aid_distributions	Related keys and constraints	SELECT	Report queries must use existing related records	Report data and summary results

Database boundary rule

In the prescribed Flutter Web–REST API–PostgreSQL stack, Flutter Web should not connect directly to PostgreSQL. The backend owns database credentials and uses the database driver/ORM or approved data-access layer. PostgreSQL remains responsible for persistent records, keys, constraints, transactions, and query execution.

10. Part F – Git/GitHub and Deployment Plan

Planning Item	What Must Be Decided	Your Project Decision / Evidence
Repository ownership	Group repository URL, members with access, and repository visibility as instructed	GitHub repository will be used to store the Flutter Web, REST API, and PostgreSQL-related project files.
Folder structure	Identify where Flutter client, backend API, documentation, and database scripts/migrations will be stored	frontend/ for Flutter Web application, backend/ for REST API source code, database/ for PostgreSQL scripts or migrations, docs/ for architecture and technology-stack blueprints, with README.md, .gitignore, and .env.example.
Branching / collaboration	Main branch, feature branches, commit/merge practice, and review responsibility	Main branch for the stable project version; feature branches for development; changes are committed and merged after checking.
Environment variables	List values that must be configured outside source code, such as API base URL and server/database secrets	API base URL, database connection settings, and server/database secrets will be stored outside the source code.
Flutter Web deployment	Hosting platform or server for the built web client	Flutter Web will be deployed to a web hosting platform selected by the team.
REST API deployment	Server, container, cloud runtime, or approved hosting platform	FastAPI backend will be deployed to a server or cloud hosting environment that supports Python.
PostgreSQL deployment	Local development instance and planned managed/server production database	PostgreSQL will be used for the local development database and a planned server/managed PostgreSQL database for deployment.
Logging / backup / recovery	Minimum plan for logs, error diagnosis, database backup, and restoration	Backend errors and important activities will be logged, while PostgreSQL data will have regular backups for recovery.

Recommended minimum repository organization (adapt as needed):

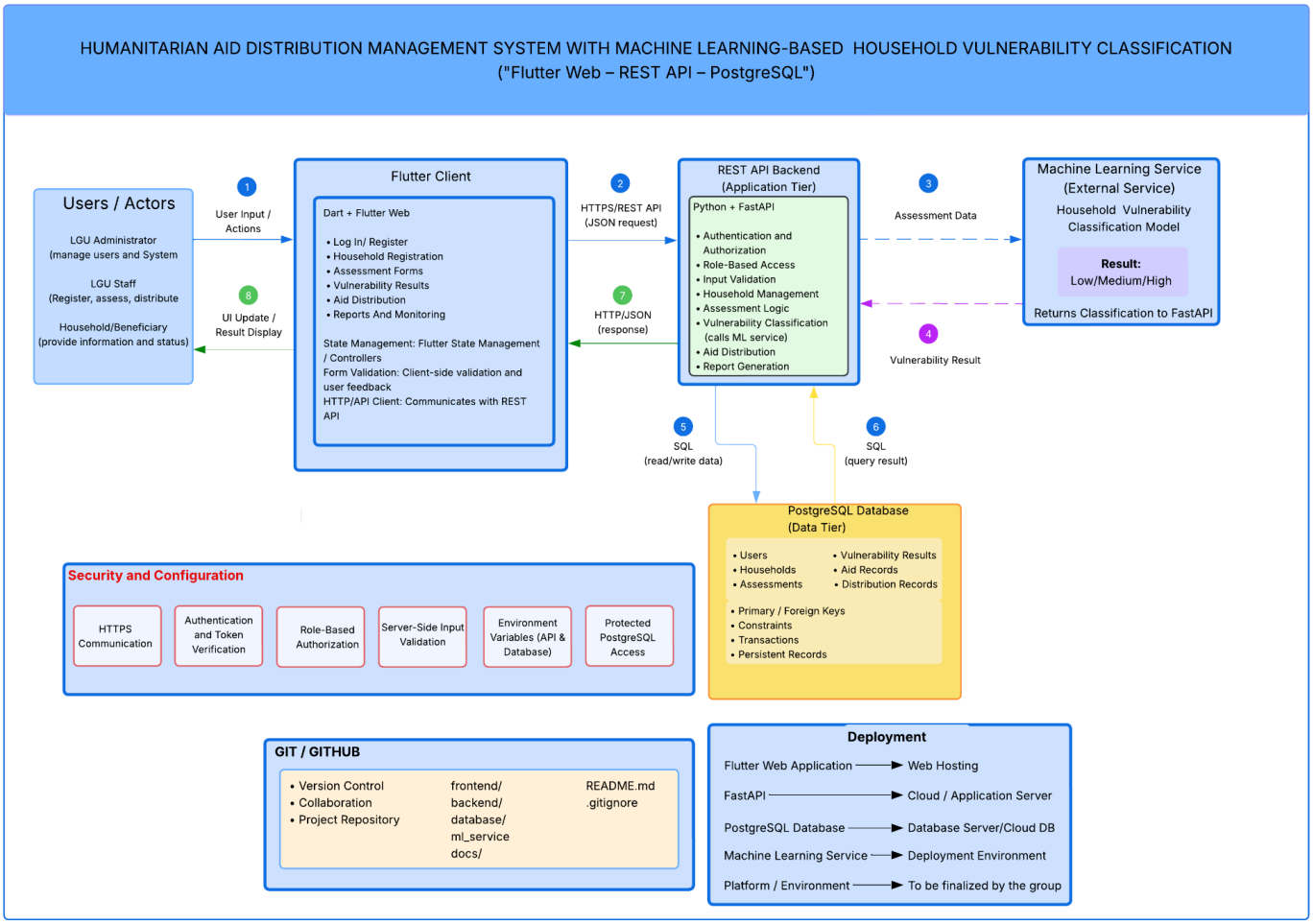
```
project-root/
  frontend/          Flutter Web application
  backend/           REST API source code
  database/          PostgreSQL scripts or migrations, when used
  docs/              architecture and technology-stack blueprints
  README.md
  .gitignore
  .env.example       variable names only; never commit real secrets
```

11. Part G – Construct the Technology Stack Blueprint

Create one clean, readable blueprint that shows how the prescribed stack is implemented in your own system. The blueprint must be understandable without opening the source code.

No.	Required Element	What Must Be Shown
1	System identity	Official project title and “Flutter Web–REST API–PostgreSQL” stack label.
2	Users / actors	Major users who interact with the Flutter Web client.

3	Flutter Web client	Dart, key screens/widgets, routing, state management, form validation, and HTTP/API client.



No.	Required Element	What Must Be Shown
4	Flutter packages	Only packages actually planned for the project; state their purpose.
5	Communication	HTTPS, REST endpoints, JSON request/response, and API base URL/environment configuration.
6	Backend framework/runtime	Exact instructor-approved framework and runtime used to implement the API.
7	Application logic	Authentication, authorization, validation, services, business rules, and external-service integration.
8	PostgreSQL persistence	Relevant tables/data domains from IT 108, keys/constraints, and database access mechanism.
9	Git/GitHub	Repository and collaboration boundary for client, backend, database scripts, and documentation.
10	Deployment	Where the Flutter build, REST API, PostgreSQL database, and storage/external services will run.
11	Security boundary	Show where secrets live, where tokens/roles are checked, and which resources are protected.
12	Data flow	Directional arrows labeled with requests, responses, JSON, SQL/data access, and returned results.

Minimum Blueprint Rule

Your blueprint must show at least one complete implementation path: User → Flutter Web interface → HTTPS/JSON REST request → backend API logic → PostgreSQL transaction/query → JSON response → Flutter state/UI update. Include Git/GitHub and deployment information as supporting infrastructure, not as steps in the runtime request path.

12. Reference Example – Do Not Copy as Your Final Output

The figure below shows the level of stack separation and labeling expected. Replace every generic label with the technologies, features, endpoints, and database objects used by your own project.

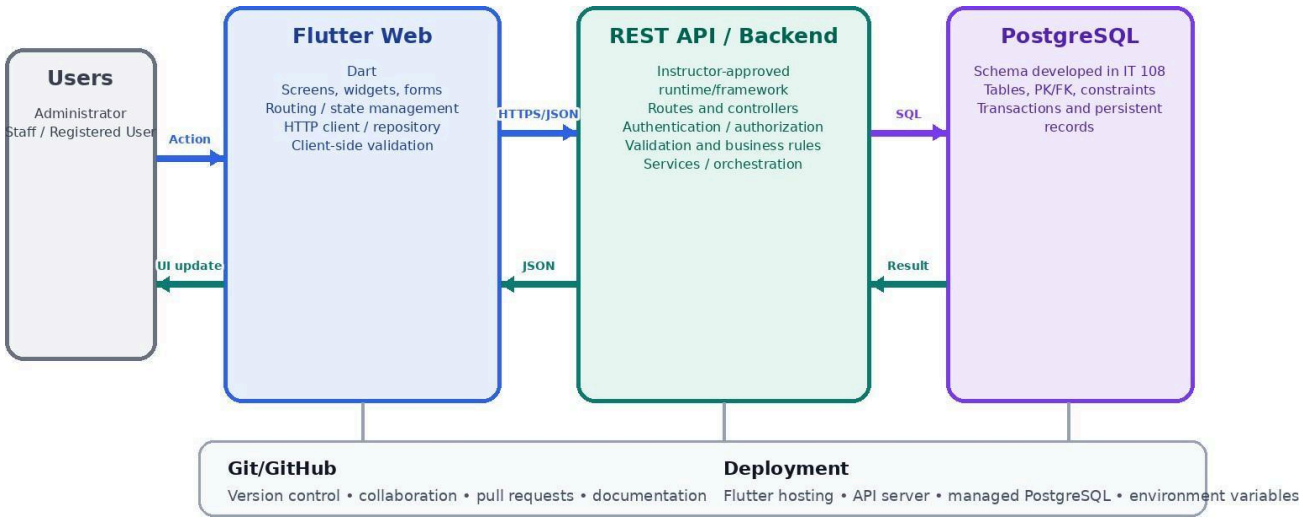


Figure 1. Reference Flutter Web–REST API–PostgreSQL technology stack and runtime path.

Step	Reference Flow
1	User interacts with a Flutter Web screen.
2	Flutter validates the form and sends an HTTPS request containing JSON.
3	The REST API verifies access, validates data, and executes business rules.
4	The backend queries or updates PostgreSQL using an approved data-access mechanism.
5	The API returns an HTTP status and JSON payload.
6	Flutter updates state and displays the result to the user.

13. Part H – Feature-to-Stack Responsibility Trace

Selected workflow / feature: Household Assessment and Vulnerability Classification

Expected result shown to the user: The household's vulnerability classification (High, Medium, or Low) is displayed in the Flutter Web application.

Step	Responsible Component	Action / Data	Why It Belongs in This Component	Evidence in Blueprint
1	Flutter Web Client	LGU Staff enters household information and assessment data.	Flutter handles user input and forms.	Flutter Web Client - Household Assessment
2	Flutter Web Client	Sends assessment data to the REST API through HTTPS/JSON.	Flutter communicates with the backend through the API.	HTTPS / JSON Request
3	Machine Learning Classification Service	Classifies the household's vulnerability level based on the assessment data.	ML Service is responsible for vulnerability classification.	ML Classification Service
4	Machine Learning Classification Service	Classifies the household's vulnerability level based on the assessment data.	ML Service is responsible for vulnerability classification.	ML Classification Service
5	FastAPI REST API	Receives the vulnerability classification and processes the result.	Backend coordinates the ML result and database operation.	FastAPI / Application Logic
6	PostgreSQL	Saves the household assessment and vulnerability classification.	PostgreSQL stores persistent system records.	PostgreSQL / Database Schema
7	FastAPI REST API	Returns the vulnerability classification to Flutter as JSON.	Backend sends the processed result back to the client.	JSON Response
8	Flutter Web Client	Updates the state and displays the vulnerability result.	Flutter handles UI updates and displays the result to the user.	State Management / Vulnerability Results

14. Part I – Stack-Role Explanation

Write 120–180 words explaining and justifying the prescribed technology stack for your own system. Address all of the following:

- What Flutter Web and Dart are responsible for in the user experience.
- Why the REST API/backend is the correct place for authentication/authorization checks, server-side validation, and business rules.
- How PostgreSQL persists the data model being developed in IT 108 and protects relational integrity.
- Why Git/GitHub and the deployment environment are necessary even though they are not runtime application layers.
- How the separation improves security, maintainability, testing, integration, and future scaling, including one implementation risk the team must manage.

Our system uses Flutter Web and Dart for the user interface, where users can log in, enter household information, submit assessments, and view results. The REST API/backend handles the important system processes like authentication, authorization, server-side validation, and business rules because these should not be handled only on the client side. PostgreSQL is used to store the system data and maintain the relationships and integrity of the database being developed in IT 108. Git and GitHub are needed for version control, collaboration, and keeping our project files organized, while the deployment environment allows the system to run outside our local computers. Separating these parts makes the system easier to maintain, test, secure, and connect with other services. It can also help if the system needs to scale or improve in the future. One risk we need to manage is keeping the communication between the Flutter client, backend, database, and ML service working properly without exposing sensitive information.

15. Final Quality Checklist

Checklist A	Checklist B
[ ] The official system title and actual project features are used.	[ ] The stack is explicitly labeled Flutter Web–REST API–PostgreSQL.
[ ] Framework, library/package, runtime, DBMS, version control, and deployment platform are classified correctly.	[ ] Flutter screens/widgets, client state, packages, and API calls are project-specific.
[ ] The instructor-approved backend framework/runtime is named.	[ ] At least six meaningful REST operations are defined.
[ ] Authentication, authorization, validation, and business rules are located primarily in the backend/API layer.	[ ] The PostgreSQL mapping matches the IT 108 schema or clearly identifies coordinated schema changes.
[ ] Flutter Web does not contain database credentials or direct raw-SQL access to PostgreSQL.	[ ] HTTPS/REST/JSON and database-access arrows are directional and meaningfully labeled.
[ ] Git/GitHub organization, environment variables, and deployment targets are identified.	[ ] At least one complete User → Flutter → API → PostgreSQL → API → Flutter path is visible.
[ ] The written stack-role explanation agrees with the blueprint and worksheets.	[ ] Every group member can explain one component and defend the complete integration path without reading a script.

16. Submission Requirements

- PDF or PNG of the final technology stack blueprint in readable landscape orientation.
- Editable blueprint source file such as .drawio, .pptx, .fig, or the native file of the selected diagramming tool.
- Completed Parts A–H and the 120–180 word stack-role explanation.
- Updated project GitHub repository containing the blueprint and supporting files under a folder such as docs/technology-stack/.
- Repository link and required files submitted through MasaoLMS using the naming convention IT110_LA3_SystemName.
- Deliver a 5–7 minute group presentation using the final blueprint as the main visual, followed by 2–3 minutes of questions. Every member must participate meaningfully.

Academic Integrity and Competency Validation

The output must represent the team's own system and technology decisions. AI tools may be used to clarify terminology or review ideas, but the final stack classification, framework/runtime selection, endpoint plan, PostgreSQL mapping, blueprint, and justification must be verified, understood, and defended by the team. The instructor may ask any member to explain why a component belongs in a particular layer or trace a request without assistance.

17. Scoring Rubric – 80 Points

Partial credit may be awarded when evidence is present but incomplete, inconsistent, or technically inaccurate. For the presentation criterion, points may be reduced for unequal participation, excessive reading, incorrect role assignment, weak integration reasoning, or inability to answer questions about the team's own stack.

Criterion	Evidence for Full Credit	Points

Criterion	Evidence for Full Credit	Points
Framework/library/runtime and stack-role accuracy	Technologies are classified correctly, and the team clearly distinguishes Flutter, Dart, packages, REST, backend runtime/framework, PostgreSQL, Git/GitHub, and deployment responsibilities.	10
Flutter Web client planning	Screens/features, widgets/forms, state approach, packages, client validation, API calls, loading/error feedback, and client security boundaries are complete and project-specific.	10
REST API and backend plan	An instructor-approved backend framework/runtime is justified; endpoints, methods, authentication/roles, validation, business rules, request data, responses, and status behavior are coherent.	14
PostgreSQL persistence integration	API resources map correctly to the IT 108 schema; important tables, PK/FK constraints, DML operations, transactions, and integrity requirements are represented accurately.	11
Technology stack blueprint and end-to-end integration	The blueprint clearly integrates users, Flutter Web, packages/state, HTTPS/REST/JSON, backend application logic, PostgreSQL, security, and a complete request-response path.	15
Git/GitHub, deployment, and secret management	Repository organization, collaboration approach, environment variables, hosting targets, and protection of server/database secrets are identified and realistic.	5
Written stack-role explanation	The 120–180 word explanation accurately justifies component roles, separation of concerns, IT 108 integration, security, maintainability, testing, deployment, scaling, and one implementation risk.	5
Group presentation and oral defense	Presentation is clear, organized, within time, and uses the final blueprint effectively; all members participate and correctly defend technology roles, API/database boundaries, and the selected workflow.	10

Raw Score	Equivalent	Checked by	Date
/ 80	_____	_____	_____

18. Post-Laboratory Reflection

Which responsibility was hardest for your group to assign correctly among Flutter Web, the REST API, and PostgreSQL, and why?

For us, the hardest part was separating the Flutter Web and REST API responsibilities. At first, it can be confusing because both handle validation and user actions. We learned that Flutter handles the forms, input, and feedback, while the REST API handles the important validation, authentication, authorization, and business rules.

Which Flutter package or client-side pattern is most important to your project, and what problem will it solve?

For our project, the Flutter State Management / Controllers pattern is important because it helps manage the data and changes on the screens. It will help us update the vulnerability result, loading status, and other information without mixing too much logic inside the UI.

What server-side rule must never rely only on Flutter client validation?

Authentication, authorization, and role checking should never rely only on Flutter validation. The REST API must check if the user is really authenticated and has the correct role before allowing protected actions.

Which PostgreSQL constraint or transaction rule from IT 108 is most important to preserving correct system data?

The primary key and foreign key relationships are important because they help keep the records connected correctly. For example, an assessment should be connected to the correct household instead of having unrelated or invalid records.

What must your team configure before the first real Flutter-to-REST-API integration test can succeed?

We need to configure the REST API server, API base URL, endpoints, and database connection first. Flutter must also be able to send HTTPS/REST requests with JSON and receive the JSON response from the backend.

What is the first stack component your group should implement after this activity, and what evidence will prove that it works?

The first component we should implement is the REST API/backend because it connects the Flutter client, PostgreSQL, and ML service. Evidence that it works would be a successful API request from Flutter and a correct JSON response from the backend.